3. Многослойный перцептрон. Кросс-валидация.

Цель работы - реализовать и исследовать многослойный перцептрон (MLP) для задач классификации и регрессии на синтетических двумерных выборках из задания 2. В первой части требуется построить модели с различными функциями активации (sigmoid, tanh, ReLU) и варьируемой архитектурой (число скрытых слоев 2–4 и число нейронов в слое 1–5), обучить их на фиксированных датасетах и оценить качество по соответствующим функциям потерь: кросс-энтропии для классификатора и MSE для регрессора. Во второй части необходимо определить достаточный размер обучающей выборки, обеспечивающий достижение точности классификации не ниже 90% на тестовых данных при фиксированных параметрах MLP, а также выполнить кросс-валидацию для выбора наилучших значений функции активации и количества слоев.

Задания:

1. Реализация и последующее обучение многослойного перцептрона
2. Проведение кросс-валидации

Ссылка на исходный код лабораторной работы:
https://github.com/F1ameX/Modern-Methods-of-Deep-Machine-Learning/blob/main/3_multilayer_perceptron/3_multilayer_perceptron.ipynb

Ход работы

Реализация многослойного перцептрона

В ходе проведения данного эксперимента был реализован многослойный перцептрон на базе модуля torch. Архитектура сети формировалась динамически по заданным гиперпараметрам: функция активации (sigmoid/tanh/ReLU), число скрытых слоев и число нейронов в слое. Для обучения использовался мини-батч градиентный спуск с оптимизатором Adam и соответствующей функцией потерь: BCELoss для бинарной классификации и MSELoss для регрессии. Рассмотрим код реализации MLP:

```python
class MultiLayerPerceptron(nn.Module):
    def __init__(self,
                 activation : str = 'sigmoid',
                 layers : int = 4,
                 neurons : int = 3,
                 task : str = 'classification'):

        super().__init__()
        assert activation in ['sigmoid', 'tanh', 'relu']
        assert task in ['classification', 'regression']
        self.task = task

        activation_function = {
            'sigmoid' : nn.Sigmoid,
            'tanh' : nn.Tanh,
            'relu' : nn.ReLU,
        }[activation]

        self.loss_function = {
            'classification' : nn.BCELoss(),
            'regression' : nn.MSELoss(),
        }[task]

        dims = [2] + [neurons] * (layers) + [1]

        modules = []
        for i in range(len(dims) - 1):
            modules.append(nn.Linear(dims[i], dims[i + 1]))
            if i != len(dims) - 2:
                modules.append(activation_function())
            else:
                if task == 'classification':
                    modules.append(nn.Sigmoid())

        self.network = nn.Sequential(*modules)

    def forward(self, X) -> torch.Tensor:
        preds = self.network(X)
        return preds
```

Рисунок 1 - Первая часть кода реализации MLP

```python
def fit(self,
        X: torch.Tensor,
        y: torch.Tensor,
        learning_rate : float = 0.01,
        n_epochs : int = 100,
        batch_size : int = 16,
        random_state : int = 42,
        ):
    rng = np.random.default_rng(seed = random_state)

    y = y.reshape(-1, 1)

    _X = X.clone()
    _y = y.clone()

    optimizer = torch.optim.Adam(self.parameters(), lr = learning_rate)

    for epoch in tqdm(
            range(n_epochs),
            desc = "Fitting MLP",
            unit = "epoch",
            total = n_epochs,
            dynamic_ncols = True,
            leave = True,
            mininterval = 0.1,
            disable = True):

        self.train()
        permutation = rng.permutation(X.shape[0])
        _X = X.clone()[permutation]
        _y = y.clone()[permutation]

        for batch_start in range(0, X.shape[0], batch_size):
            _X_batch = _X[batch_start : batch_start + batch_size]
            _y_batch = _y[batch_start : batch_start + batch_size]

            preds = self(_X_batch)
            loss = self.loss_function(preds, _y_batch)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

    return self

def predict(self, X: torch.Tensor) -> torch.Tensor:
    self.eval()
    with torch.no_grad():

        if self.task == 'classification':
            prob = self(X)
            pred = (prob >= 0.5).to(torch.float64)
            return pred

        elif self.task == 'regression':
            pred = self(X)
            return pred.to(torch.float64)
```

Рисунок 2 - Вторая часть кода реализации MLP

После этого модель обучалась на синтетически сгенерированных выборках из задания 2 в режиме перебора гиперпараметров: для каждой функции активации (sigmoid, tanh, ReLU) последовательно рассматривались различные архитектуры с числом скрытых слоев от 2 до 4 и числом нейронов в слое от 1 до 5. Для каждого набора параметров модель обучалась на фиксированном разбиении train/test в пропорции 75/25 (train = 375 объектов, test = 125 объектов), а полученные метрики (accuracy для классификации и MSE для регрессии) заносились в таблицы для последующего сравнения. Рассмотрим полученные таблицы для классификации:

========== Концентрические окружности ==========

Таблица accuracy для функции активации sigmoid

Neurons	1	2	3	4	5
Layers					
2	0.664	0.84	0.712	1.0	1.0
3	0.664	0.88	1.0	1.0	1.0
4	0.648	0.84	1.0	1.0	1.0

Таблица accuracy для функции активации tanh

Neurons	1	2	3	4	5
Layers					
2	0.72	0.88	1.0	1.0	1.0
3	0.672	0.832	1.0	1.0	1.0
4	0.496	0.88	1.0	1.0	1.0

Таблица accuracy для функции активации relu

Neurons	1	2	3	4	5
Layers					
2	0.72	0.88	1.0	1.0	1.0
3	0.496	0.952	1.0	1.0	1.0
4	0.496	0.936	0.496	1.0	1.0

Рисунок 3 - Таблицы Accuracy для концентрических окружностей

========== Гауссовские кластеры ==========

Таблица accuracy для функции активации sigmoid

Neurons / Layers	1	2	3	4	5
2	0.984	0.984	0.976	0.976	0.976
3	0.984	0.984	0.984	0.976	0.976
4	0.984	0.968	0.984	0.976	0.976

Таблица accuracy для функции активации tanh

Neurons / Layers	1	2	3	4	5
2	0.984	0.984	0.984	0.976	0.976
3	0.984	0.976	0.976	0.976	0.976
4	0.984	0.992	0.976	0.976	0.984

Таблица accuracy для функции активации relu

Neurons / Layers	1	2	3	4	5
2	0.432	0.984	0.984	0.968	0.976
3	0.432	0.984	0.432	0.976	0.968
4	0.432	0.432	0.984	0.976	0.976

Рисунок 4 - Таблицы Accuracy для гауссовских кластеров

========== XOR распределение ==========

Таблица accuracy для функции активации sigmoid

Neurons / Layers	1	2	3	4	5
2	0.672	0.944	0.688	1.0	1.0
3	0.664	0.752	0.992	0.688	0.992
4	0.736	0.744	0.96	1.0	0.992

Таблица accuracy для функции активации tanh

Neurons / Layers	1	2	3	4	5
2	0.776	0.96	1.0	0.992	0.992
3	0.784	0.688	1.0	1.0	1.0
4	0.728	0.752	1.0	1.0	1.0

Таблица accuracy для функции активации relu

Neurons / Layers	1	2	3	4	5
2	0.744	0.752	1.0	0.688	1.0
3	0.44	0.976	0.992	1.0	1.0
4	0.72	0.44	0.68	1.0	1.0

Рисунок 5 - Таблицы Accuracy для XOR распределения

```
========================= Спирали Архимеда =========================
Таблица accuracy для функции активации sigmoid
```

Neurons	1	2	3	4	5
Layers					
2	0.432	0.432	0.432	0.424	0.496
3	0.432	0.52	0.432	0.432	0.432
4	0.504	0.504	0.504	0.512	0.424

```
Таблица accuracy для функции активации tanh
```

Neurons	1	2	3	4	5
Layers					
2	0.432	0.432	0.52	0.48	0.496
3	0.432	0.432	0.44	0.472	0.536
4	0.432	0.432	0.432	0.536	0.496

```
Таблица accuracy для функции активации relu
```

Neurons	1	2	3	4	5
Layers					
2	0.512	0.52	0.456	0.528	0.48
3	0.432	0.432	0.496	0.544	0.464
4	0.432	0.432	0.48	0.504	0.584

Рисунок 6 - Таблицы Accuracy для Спиралей Архимеда

По таблицам accuracy видно: blobs классифицируются проще всего - почти любые архитектуры дают высокий результат, circles и XOR требуют уже “не совсем маленькой” сети (2–4 скрытых слоя, 3–5 нейронов), а спирали остаются самыми сложными — даже при максимальных параметрах точность заметно ниже из-за сильно нелинейной границы.

Далее переходим к режиму регрессии, где качество оценивается по MSE на тестовой выборке для тех же наборов гиперпараметров:

========== Концентрические окружности ==========

Таблица MSE для функции активации sigmoid

Neurons / Layers	1	2	3	4	5
2	0.207	0.125	0.006	0.007	0.01
3	0.205	0.202	0.0	0.001	0.0
4	0.192	0.122	0.0	0.205	0.0

Таблица MSE для функции активации tanh

Neurons / Layers	1	2	3	4	5
2	0.193	0.1	0.001	0.001	0.006
3	0.202	0.126	0.007	0.0	0.0
4	0.186	0.094	0.0	0.0	0.0

Таблица MSE для функции активации relu

Neurons / Layers	1	2	3	4	5
2	0.202	0.126	0.0	0.004	0.0
3	0.25	0.127	0.25	0.0	0.0
4	0.25	0.115	0.25	0.001	0.0

Рисунок 7 - таблицы MSE для концентрических окружностей

========== Гауссовские кластеры ==========

Таблица MSE для функции активации sigmoid

Neurons / Layers	1	2	3	4	5
2	0.012	0.011	0.012	0.011	0.012
3	0.014	0.012	0.012	0.013	0.016
4	0.014	0.014	0.014	0.017	0.031

Таблица MSE для функции активации tanh

Neurons / Layers	1	2	3	4	5
2	0.014	0.014	0.013	0.015	0.013
3	0.014	0.015	0.025	0.018	0.009
4	0.015	0.019	0.022	0.024	0.027

Таблица MSE для функции активации relu

Neurons / Layers	1	2	3	4	5
2	0.253	0.007	0.02	0.032	0.009
3	0.251	0.011	0.016	0.014	0.021
4	0.254	0.253	0.015	0.032	0.011

Рисунок 8 - таблицы MSE для гауссовских кластеров

========== XOR распределение ==========

Таблица MSE для функции активации sigmoid

Neurons / Layers	1	2	3	4	5
2	0.165	0.038	0.153	0.001	0.002
3	0.185	0.038	0.0	0.0	0.004
4	0.199	0.148	0.008	0.008	0.0

Таблица MSE для функции активации tanh

Neurons / Layers	1	2	3	4	5
2	0.185	0.038	0.0	0.0	0.0
3	0.185	0.151	0.0	0.0	0.0
4	0.167	0.036	0.0	0.002	0.0

Таблица MSE для функции активации relu

Neurons / Layers	1	2	3	4	5
2	0.166	0.174	0.002	0.0	0.0
3	0.166	0.253	0.016	0.0	0.0
4	0.201	0.154	0.0	0.148	0.0

Рисунок 9 - таблицы MSE для XOR распределения

========== Спирали Архимеда ==========

Таблица MSE для функции активации sigmoid

Neurons / Layers	1	2	3	4	5
2	0.262	0.258	0.269	0.26	0.27
3	0.266	0.266	0.266	0.266	0.266
4	0.266	0.256	0.257	0.255	0.255

Таблица MSE для функции активации tanh

Neurons / Layers	1	2	3	4	5
2	0.262	0.262	0.27	0.28	0.268
3	0.248	0.263	0.295	0.275	0.252
4	0.256	0.271	0.269	0.226	0.268

Таблица MSE для функции активации relu

Neurons / Layers	1	2	3	4	5
2	0.251	0.259	0.26	0.259	0.282
3	0.251	0.251	0.277	0.268	0.259
4	0.251	0.253	0.253	0.263	0.251

Рисунок 10 - Таблицы MSE для спиралей Архимеда

По таблицам MSE видно, что для circles и XOR наилучшие значения достигаются уже при умеренной сложности сети (≈ 2–4 скрытых слоя и 3–5 нейронов), причём при таких конфигурациях ошибка часто падает практически до нуля, что означает очень точное приближение целевой зависимости на тесте.

Для blobs MSE в целом малое и стабильное у большинства конфигураций (задача проще, структура ближе к “линейной”), а у spirals MSE заметно выше и хуже улучшается — распределение самое сложное, и в выбранной сетке гиперпараметров регрессионная постановка остаётся трудной.

Кросс-валидация

```
def kfold_indices(n_samples: int, k: int = 5, seed: int = 52):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(n_samples)
    folds = np.array_split(idx, k)
    return folds
```

Рисунок 11 - Код реализации кросс-валидации

В ходе проведения данной части эксперимента была оценена минимальная доля обучающей выборки, достаточная для достижения качества классификации не ниже 90% на тестовой выборке при фиксированной архитектуре многослойного перцептрона. Для каждого распределения из задания 2 данные генерировались фиксированным генератором, затем выполнялось разбиение на обучающую и тестовую части. Доля обучающей выборки последовательно увеличивалась (от 5% до 95% с шагом 5%), и для каждой доли обучалась модель с одинаковыми гиперпараметрами. Минимальной считалась первая доля, при которой итоговая точность на тестовой выборке удовлетворяла условию ≥ 90%. Результаты представлены в таблице:

dataset	best_train_frac	best_train_size	target_acc
circles	0.15	75.0	0.9
blobs	0.05	25.0	0.9
xor	0.05	25.0	0.9
spiral	NaN	NaN	0.9

Рисунок 12 - Таблица с долями выборок по порогам

Для распределений blobs и xor достаточно 5% обучающих данных (25 объектов из 500) для достижения требуемой точности. Для circles потребовалась большая обучающая доля — 15% (75 объектов из 500), что объясняется более сложной границей раздела. Для spiral при выбранной фиксированной архитектуре достичь порога 0.9 не удалось ни при одной из рассмотренных долей обучающей выборки, поэтому в таблице указано значение NaN, что означает, что порог недостижим в рамках заданных настроек модели/обучения.

После этого была проведена кросс-валидация по схеме k-fold, с параметром k = 5. Для каждого датасета перебрались функции активации и число скрытых слоев при фиксированном числе нейронов равном 5. Для каждой конфигурации считались метрики средняя точность cv_mean_acc, стандартное отклонение cv_std_acc и минимальная точность cv_min_acc по 5 фолдам, после чего выбиралась конфигурация с наилучшим качеством.

```
===== CIRCLES =====
activation=sigmoid, layers=2, neurons=5 -> mean=1.0000, std=0.0000, min=1.0000
activation=sigmoid, layers=3, neurons=5 -> mean=0.8600, std=0.1918, min=0.6400
activation=sigmoid, layers=4, neurons=5 -> mean=1.0000, std=0.0000, min=1.0000
activation=tanh, layers=2, neurons=5 -> mean=0.9980, std=0.0045, min=0.9900
activation=tanh, layers=3, neurons=5 -> mean=1.0000, std=0.0000, min=1.0000
activation=tanh, layers=4, neurons=5 -> mean=1.0000, std=0.0000, min=1.0000
activation=relu, layers=2, neurons=5 -> mean=1.0000, std=0.0000, min=1.0000
activation=relu, layers=3, neurons=5 -> mean=1.0000, std=0.0000, min=1.0000
activation=relu, layers=4, neurons=5 -> mean=0.8940, std=0.2370, min=0.4700

===== BLOBS =====
activation=sigmoid, layers=2, neurons=5 -> mean=0.9920, std=0.0045, min=0.9900
activation=sigmoid, layers=3, neurons=5 -> mean=0.9920, std=0.0045, min=0.9900
activation=sigmoid, layers=4, neurons=5 -> mean=0.9900, std=0.0071, min=0.9800
activation=tanh, layers=2, neurons=5 -> mean=0.9880, std=0.0084, min=0.9800
activation=tanh, layers=3, neurons=5 -> mean=0.9880, std=0.0084, min=0.9800
activation=tanh, layers=4, neurons=5 -> mean=0.9860, std=0.0114, min=0.9700
activation=relu, layers=2, neurons=5 -> mean=0.9880, std=0.0084, min=0.9800
activation=relu, layers=3, neurons=5 -> mean=0.9880, std=0.0110, min=0.9800
activation=relu, layers=4, neurons=5 -> mean=0.9900, std=0.0100, min=0.9800

===== XOR =====
activation=sigmoid, layers=2, neurons=5 -> mean=0.9920, std=0.0084, min=0.9800
activation=sigmoid, layers=3, neurons=5 -> mean=0.9960, std=0.0055, min=0.9900
activation=sigmoid, layers=4, neurons=5 -> mean=0.9380, std=0.1277, min=0.7100
activation=tanh, layers=2, neurons=5 -> mean=0.9960, std=0.0055, min=0.9900
activation=tanh, layers=3, neurons=5 -> mean=0.9960, std=0.0055, min=0.9900
activation=tanh, layers=4, neurons=5 -> mean=0.9940, std=0.0089, min=0.9800
activation=relu, layers=2, neurons=5 -> mean=0.9440, std=0.1197, min=0.7300
activation=relu, layers=3, neurons=5 -> mean=0.9960, std=0.0089, min=0.9800
activation=relu, layers=4, neurons=5 -> mean=0.9960, std=0.0055, min=0.9900

===== SPIRAL =====
activation=sigmoid, layers=2, neurons=5 -> mean=0.4700, std=0.0292, min=0.4300
activation=sigmoid, layers=3, neurons=5 -> mean=0.4420, std=0.0217, min=0.4100
activation=sigmoid, layers=4, neurons=5 -> mean=0.5480, std=0.0522, min=0.4800
activation=tanh, layers=2, neurons=5 -> mean=0.4840, std=0.0550, min=0.4200
activation=tanh, layers=3, neurons=5 -> mean=0.4800, std=0.0587, min=0.4000
activation=tanh, layers=4, neurons=5 -> mean=0.6060, std=0.0702, min=0.5300
activation=relu, layers=2, neurons=5 -> mean=0.4940, std=0.0811, min=0.4000
activation=relu, layers=3, neurons=5 -> mean=0.5620, std=0.0881, min=0.4600
activation=relu, layers=4, neurons=5 -> mean=0.6140, std=0.0615, min=0.5100
```

Рисунок 13 - Подробные результаты кросс-валидации

dataset	best_activation	best_layers	neurons_fixed	cv_mean_acc	cv_std_acc	cv_min_acc	k_folds
circles	sigmoid	2	5	1.000	0.000000	1.00	5
blobs	sigmoid	2	5	0.992	0.004472	0.99	5
xor	sigmoid	3	5	0.996	0.005477	0.99	5
spiral	relu	4	5	0.614	0.061482	0.51	5

Рисунок 14 - Структурированные результаты кросс-валидации

По итоговой сводной таблице лучшими оказались:

- Концентрические окружности: sigmoid, 2 слоя
- Гауссовские кластеры: sigmoid, 2 слоя
- XOR распределение: sigmoid, 3 слоя
- Спирали Архимеда: ReLU, 4 слоя

Таким образом, для circles/blobs/xor большинство конфигураций дают высокую и стабильную точность (малый разброс по фолдам), тогда как для spiral качество существенно ниже и заметно более нестабильно, что отражает более сложную (нелинейную) структуру границы классов для данного распределения.

Выводы

В первой части перебором архитектур видно, что circles / blobs / xor решаются уверенно: при достаточной нелинейности сети (примерно 2–4 скрытых слоя и 3–5 нейронов) accuracy быстро выходит к ~1.0 на тесте почти для всех активаций. Blobs — самые “легкие”, circles требуют чуть большей ёмкости из-за более сложной границы, xor стабильно решается только при достаточно “богатой” архитектуре (что ожидаемо). Spiral оказался самым сложным: даже при усложнении сети качество заметно ниже и до уровня остальных распределений не дотягивает.

Во второй части для фиксированной архитектуры оценили минимальную долю train для accuracy ≥ 0.9: blobs — 5% (25/500), xor — 5% (25/500), circles — 15% (75/500); для spiral порог 0.9 не достигнут ни при одной доле (NaN). По 5-fold CV лучшие настройки получились: circles — sigmoid + 2 слоя, blobs

— sigmoid + 2 слоя, xor — sigmoid + 3 слоя, spiral — relu + 4 слоя, но у spiral средняя точность всё равно низкая и разброс выше. По итогу получили, что MLP в заданных пределах гиперпараметров хорошо работает на circles/blobs/xor, а spiral требует более мощной/иначе настроенной модели.